pro01-Nginx-Python-Flask-FastAPI-MySQL

Documentacion del proyecto

Aplicacion web de reservas de hoteles

Arquitectura: Nginx + Flask (Python) + MySQL + phpMyAdmin

Frontend: HTML5 + CSS3 + JavaScript vanilla

Despliegue: Docker Compose

Indice

1. Estructura general del proyecto
2. docker-compose.yml - Orquestador de servicios
3. backend/ - API REST en Python (Flask)
4. db/ - Base de datos MySQL
5. frontend/ - Interfaz de usuario
6. nginx/ - Servidor web y proxy inverso
7. Flujo completo paso a paso
8. Pagina principal - index.html y boton Buscar
9. Resultados de busqueda - resultados.html
10. Detalle del hotel y reserva - detalle.html

1. Estructura general del proyecto

Este proyecto es una aplicacion web completa de reservas de hoteles que se despliega con Docker Compose. Cuenta con 4 servicios: una base de datos MySQL para almacenar la informacion, una API REST en Python con Flask como backend, un frontend estatico servido por Nginx, y phpMyAdmin para administrar la base de datos visualmente.

Estructura de directorios:

```
pro01-Nginx-Python-Flask-FastAPI-MySQL/
  +-- backend/          # API REST en Flask (Python)
  |   +-- main.py       # Logica de la API (264 lineas)
  |   +-- requirements.txt
  |   +-- Dockerfile
  +-- db/               # Scripts SQL de la base de datos
  |   +-- init.sql      # Inicializacion (tablas, datos, SP)
  |   +-- datos_reservas.sql
  |   +-- update.sql
  +-- frontend/         # Interfaz de usuario
  |   +-- index.html    # Pagina principal
  |   +-- resultados.html
  |   +-- detalle.html
  |   +-- css/style.css
  |   +-- js/           # Logica del frontend
  +-- nginx/
  |   +-- default.conf  # Configuracion del servidor
  +-- docker-compose.yml # Orquestador de servicios
```

2. docker-compose.yml - Orquestador de servicios

El archivo docker-compose.yml define y conecta 4 contenedores que trabajan juntos para formar la aplicacion completa:

Servicios definidos:

2.1. Servicio: db (MySQL 8.0)

Contenedor: db-mysql. Usa la imagen oficial mysql:8.0. Monta el archivo ./db/init.sql en /docker-entrypoint-initdb.d/ para que se ejecute automaticamente al arrancar por primera vez, creando las tablas, el trigger, el Stored Procedure e insertando 6 hoteles de ejemplo. Expone el puerto 3306. Tiene un healthcheck con mysqladmin ping para que el backend espere a que MySQL este listo.

2.2. Servicio: backend (Flask API)

Contenedor: backend-flask. Construye la imagen desde ./backend usando su Dockerfile. Monta el codigo en /app para desarrollo en caliente. Recibe por variables de entorno los datos de conexion a MySQL (DB_HOST, DB_USER, DB_PASS, DB_NAME). Depende del servicio db y espera a que el healthcheck de MySQL pase. Expone el puerto 5000.

2.3. Servicio: nginx

Contenedor: servidor-nginx. Usa la imagen oficial nginx:alpine. Expone el puerto 80. Monta la carpeta ./frontend como raiz web (/usr/share/nginx/html) y el archivo ./nginx/default.conf como configuracion del servidor. Depende del backend y espera a que su healthcheck pase.

2.4. Servicio: phpmyadmin

Contenedor: pma-control. Usa la imagen phpmyadmin:5.2. Expone el puerto 8081. Se conecta automaticamente al servicio db mediante la variable PMA_HOST. Permite gestionar la base de datos MySQL desde un navegador web.

3. backend/ - API REST en Python (Flask)

El backend es una API REST desarrollada en Python con el framework Flask. Se encarga de recibir peticiones del frontend, consultar la base de datos MySQL y devolver respuestas en formato JSON. Tambien contiene toda la logica de negocio de las reservas.

Archivos:

main.py (264 lineas)

Contiene la aplicacion Flask con los siguientes endpoints:

```
GET  /health            - Healthcheck del backend
GET  /api/test-db       - Prueba de conexion a MySQL
GET  /api/hoteles/destacados - Devuelve los primeros 6 hoteles
GET  /api/hoteles       - Busca hoteles con filtros (ciudad, fechas, hab.)
GET  /api/hoteles/<id> - Detalle de un hotel + disponibilidad
POST /api/reservas      - Crea una nueva reserva
```

Funcionalidades del backend: CORS habilitado para permitir peticiones desde el frontend, manejo de errores global (404 y 500), sanitizacion de datos HTML para seguridad, validacion de fechas (que la salida sea posterior a la entrada, que no sean anteriores a hoy), calculo de disponibilidad de habitaciones consultando reservas existentes, y llamada a un Stored Procedure para crear reservas de forma atomica.

requirements.txt

Lista las dependencias de Python necesarias: flask (framework web), flask-cors (para permitir CORS) y mysql-connector-python (conector oficial de MySQL).

Dockerfile

Imagen basada en python:3.11-slim. Crea un usuario no root 'appuser' por seguridad, copia e instala las dependencias, copia el codigo, expone el puerto 5000 y ejecuta main.py como comando de inicio.

4. db/ - Base de datos MySQL

La carpeta db contiene los scripts SQL que definen la estructura y los datos de la base de datos MySQL llamada 'bd1'.

Archivos:

init.sql

Script de inicializacion que se ejecuta al arrancar el contenedor MySQL por primera vez. Contiene:

- Tabla 'hoteles': Almacena la informacion de cada hotel (id, nombre, ciudad, descripcion, precio por noche, numero de habitaciones, imagenes).
- Tabla 'reservas': Almacena las reservas realizadas (id, hotel_id, nombre del cliente, fecha de entrada, fecha de salida, numero de habitaciones).
- Tabla 'auditoria_reservas': Registro historico de todas las reservas creadas, con fecha y hora.
- Trigger 'tr_auditoria_reservas': Se ejecuta automaticamente tras cada insercion en la tabla reservas, registrando la accion en auditoria_reservas.
- Stored Procedure 'sp_realizar_reserva': Procedimiento almacenado que verifica la disponibilidad de habitaciones y crea la reserva si hay sitio, o devuelve un mensaje de error si no.
- Insercion de 6 hoteles de ejemplo en ciudades espanolas (Madrid, Barcelona, Sevilla, Valencia, Granada, Bilbao) con datos realistas (precios, descripciones, imagenes de Unsplash).

datos_reservas.sql

Script auxiliar para exportar/importar datos de reservas entre entornos. Contiene un ejemplo de reserva y su registro de auditoria.

update.sql

Script de actualizacion que contiene las mismas definiciones de tablas, trigger y Stored Procedure que init.sql, pero ademas incluye correcciones de URLs de imagenes para los hoteles. Puede ejecutarse si se necesita actualizar la BD sin reiniciarla.

5. frontend/ - Interfaz de usuario

El frontend es una aplicacion web estatica (sin framework) construida con HTML5, CSS3 y JavaScript vanilla. Se sirve directamente desde Nginx y se comunica con el backend a traves de peticiones HTTP a la API.

Paginas HTML:

index.html - Pagina principal

- Formulario de busqueda con 4 campos: ciudad (texto), fecha de entrada y salida (date), numero de habitaciones (number). Incluye validacion basica en cliente.
- Seccion 'Hoteles destacados' con un grid de tarjetas que se cargan dinamicamente desde la API.
- Incluye los archivos JS: buscador.js y destacados.js.

resultados.html - Resultados de busqueda

- Titulo dinamico: 'Hoteles en [ciudad]'.
- Grid de tarjetas de hotel con imagen, nombre, ciudad, descripcion, precio por noche y disponibilidad.
- Muestra contador de resultados o mensaje 'No se encontraron hoteles'.
- Incluye resultados.js.

detalle.html - Detalle del hotel y formulario de reserva

- Imagen grande del hotel, nombre, ciudad, descripcion.
- Precio por noche, total de habitaciones y habitaciones disponibles.
- Formulario de reserva con campos: nombre completo, fecha de entrada, fecha de salida, numero de habitaciones.
- Resumen de precio en tiempo real que se actualiza al cambiar fechas o habitaciones (precio x noches x habitaciones).
- Mensajes de error o exito visibles sin recargar la pagina.
- Incluye detalle.js.

Archivos JavaScript:

buscador.js

Gestiona el formulario de busqueda en index.html. Establece la fecha minima de los campos date al dia actual. Al enviar el formulario, valida que todos los campos esten rellenos y que la fecha de salida sea posterior a la de entrada. Si es valido, redirige a resultados.html con los parametros en la URL.

destacados.js

Al cargar index.html, llama a GET /api/hoteles/destacados. Si la respuesta es exitosa, genera tarjetas enlazables a detalle.html para cada hotel destacado con su imagen, nombre, ciudad y precio.

resultados.js

Lee los parametros de la URL (ciudad, fechas, habitaciones) y llama a GET /api/hoteles con esos filtros. Si hay resultados, los pinta como tarjetas enlazables a detalle.html. Si no hay resultados, muestra un mensaje de 'no encontrados'. En caso de error del servidor, muestra un mensaje de error.

detalle.js

Es el archivo mas complejo (266 lineas). Al cargar detalle.html, lee el id del hotel y las fechas de la URL, y llama a GET /api/hoteles/<id> para obtener todos los datos. Rellena la pagina con la informacion del hotel.

Gestiona el formulario de reserva con: validacion visual de cada campo, calculo del resumen de precio en tiempo real (precio por noche x noches x habitaciones), y envio de la reserva mediante POST /api/reservas. Tambien valida que no se exceda la disponibilidad de habitaciones. Al crear la reserva, si es exitosa recarga el detalle para reflejar la nueva disponibilidad, y si falla muestra el mensaje de error.

app.js

Pequeño script de prueba que llama a GET /api/test-db para verificar la conexion entre el frontend y la base de datos. Muestra el resultado en pantalla.

CSS:

style.css

Hoja de estilos unica para toda la aplicacion. Define el diseno responsivo, los colores, tipografia, animaciones, tarjetas de hoteles, formularios y componentes de la interfaz.

6. nginx/ - Servidor web y proxy inverso

Nginx actua como servidor web para los archivos estaticos del frontend y como proxy inverso para las peticiones a la API del backend.

default.conf

Configuracion del servidor Nginx que incluye:

- Escucha en el puerto 80.
- Compresion gzip activada para CSS, JavaScript, HTML y SVG, con un minimo de 256 bytes.
- Cabeceras de seguridad: X-Content-Type-Options (nosniff), X-Frame-Options (DENY), X-XSS-Protection, Referrer-Policy.
- Servicio de archivos estaticos desde /usr/share/nginx/html (la carpeta frontend montada).
- Control de cache: sin cache para CSS y JS, 7 dias de cache para imagenes, sin almacenamiento para index.html.
- Proxy inverso: cualquier ruta que empiece por /api/ se redirige a http://backend:5000, anadiendo las cabeceras originales del cliente (Host, X-Real-IP, X-Forwarded-For, X-Forwarded-Proto). Timeout de lectura de 30 segundos.

7. Flujo completo paso a paso

A continuacion se explica detalladamente que ocurre cuando el usuario interactua con cada elemento de la pagina web.

Diagrama de navegacion:

```
index.html
   |
   +-- (ver hoteles destacados) --> carga automatica al entrar
   |
   +-- (clic en hotel destacado) --> detalle.html?id=X
   |
   +-- (rellenar formulario y pulsar Buscar) --> resultados.html?ciudad=...&fechas=...
                                                        |
                                                        +-- (clic en hotel) --> detalle.html?id=X
                                                                                |
                                                                                +-- (rellenar formular
```

8. Pagina principal - index.html y boton Buscar

Cuando entras a http://localhost, Nginx sirve el archivo index.html. La pagina se compone de dos secciones principales:

A) Hoteles destacados (carga automatica)

Nada mas entrar en la pagina, el archivo destacados.js se ejecuta y realiza una peticion GET a /api/hoteles/destacados. El backend Flask ejecuta una consulta SQL que devuelve los primeros 6 hoteles ordenados por nombre. Con esos datos, JavaScript genera 6 tarjetas visuales con imagen, nombre, ciudad y precio.

Cada tarjeta es un enlace HTML <a> que apunta a detalle.html con los parametros del hotel y fechas por defecto (hoy y manana). Al hacer clic en cualquier hotel destacado, el navegador navega a detalle.html?id=X&fecha_entrada=...&fecha_salida=...&habitaciones=1.

B) Formulario de busqueda y boton Buscar

El formulario tiene 4 campos:

- Ciudad: campo de texto donde escribes, por ejemplo, 'Madrid'.
- Fecha de entrada: selector de fecha (date). La fecha minima es hoy (configurada por JavaScript).
- Fecha de salida: selector de fecha. La fecha minima es hoy (configurada por JavaScript).
- Habitaciones: campo numerico con valor minimo 1 y valor por defecto 1.

Al hacer clic en el boton 'Buscar', el archivo buscador.js:

- Previene el envio tradicional del formulario (e.preventDefault()).
- Valida que todos los campos tengan valor. Si falta alguno, muestra una alerta 'Todos los campos son obligatorios.'
- Valida que la fecha de salida sea posterior a la de entrada. Si no, muestra alerta 'La fecha de salida debe ser posterior a la de entrada.'
- Si todo es correcto, construye una URL con los parametros: resultados.html?ciudad=Madrid&fecha_entrada=2026-06-01&fecha_salida=2026-06-05&habitaciones=2
- Redirige el navegador a resultados.html con esos parametros en la URL.

9. Resultados de busqueda - resultados.html

Llegas a esta pagina despues de rellenar el formulario de busqueda y pulsar 'Buscar' en index.html. La URL contiene los filtros como parametros.

Al cargar la pagina:

El archivo resultados.js realiza las siguientes acciones:

Paso 1: Lectura de parametros
Lee ciudad, fecha_entrada, fecha_salida y habitaciones de la URL. Si falta alguno, redirige a la pagina principal (index.html).

Paso 2: Actualizacion del titulo
Cambia el titulo a 'Hoteles en [ciudad]'.

Paso 3: Peticion a la API
Llama a GET /api/hoteles?ciudad=Madrid&fecha_entrada=2026-06-01&fecha_salida=2026-06-05&habitaciones=2. La peticion llega a Nginx, que al ver que empieza por /api/ la redirige al backend Flask en http://backend:5000.

Paso 4: Logica del backend
Flask recibe los parametros, valida que todos esten presentes y tengan formato correcto. Luego ejecuta una consulta SQL que: busca hoteles cuya ciudad coincida (con LIKE), cruza con las reservas existentes para esas fechas calculando cuantas habitaciones estan ocupadas, y solo devuelve hoteles que tengan suficientes libres.

Ejemplo: Si el hotel tiene 10 habitaciones y ya hay 3 reservadas para esas fechas, quedan 7 disponibles. Si pides 2, el hotel aparece. Si pides 8, no aparece.

Paso 5: Renderizado de resultados
Cuando la respuesta llega al frontend, se generan tarjetas para cada hotel con: imagen, nombre, descripcion, precio por noche y disponibilidad. Cada tarjeta es un enlace a detalle.html?id=X&fecha_entrada=...&fecha_salida=...&habitaciones=..., manteniendo las fechas de la busqueda original.

Si no hay resultados, se muestra el mensaje 'No se encontraron hoteles'.

Si hay error de conexion con el servidor, se muestra 'Error al conectar con el servidor'.

Accion del usuario: clic en un hotel
Al hacer clic en cualquier tarjeta de hotel, el navegador navega a detalle.html con el id del hotel y las fechas de busqueda.

10. Detalle del hotel y reserva - detalle.html

Esta es la pagina mas importante de la aplicacion. Muestra la informacion completa de un hotel y permite realizar la reserva. Puedes llegar aqui desde los destacados o desde los resultados de busqueda.

Al cargar la pagina:

El archivo detalle.js (266 lineas) ejecuta las siguientes acciones:

Paso 1: Obtener el ID del hotel
Lee el parametro 'id' de la URL. Si no existe, redirige a la pagina principal.

Paso 2: Peticion a la API
Llama a GET /api/hoteles/<id>?fecha_entrada=...&fecha_salida=... Flask consulta la base de datos, obtiene todos los datos del hotel y calcula cuantas habitaciones estan ocupadas para el rango de fechas solicitado.

Paso 3: Rellenar la pagina
Con los datos recibidos, se actualizan: la imagen grande, el nombre del hotel, la ciudad, la descripcion, el precio por noche, el total de habitaciones y las habitaciones disponibles. Tambien se rellenan los campos del formulario con las fechas y numero de habitaciones recibidos de la pagina anterior.

Resumen de precio en tiempo real

Debajo del formulario hay un resumen que se actualiza automaticamente al cambiar cualquier campo. La funcion actualizarResumen() calcula:

- Precio por noche (fijo del hotel).
- Numero de noches (diferencia entre fecha de salida y entrada).
- Numero de habitaciones solicitadas.
- Total = precio por noche x noches x habitaciones.

El calculo se dispara con los eventos 'change' en las fechas y 'input' en habitaciones.

Formulario de reserva - Campos:

- Nombre completo: campo de texto obligatorio (maximo 100 caracteres).
- Fecha de entrada: selector de fecha. Si vienes de una busqueda, ya viene rellena.
- Fecha de salida: selector de fecha. Si vienes de una busqueda, ya viene rellena.
- Habitaciones: numero de habitaciones a reservar, minimo 1.

Boton: Confirmar reserva

Al hacer clic en 'Confirmar reserva', detalle.js ejecuta:

Fase 1: Validacion en cliente

- El nombre no puede estar vacio.
- Las fechas deben estar seleccionadas.
- Las habitaciones deben ser al menos 1.
- La fecha de salida debe ser posterior a la de entrada.
- La fecha de entrada no puede ser anterior a hoy.
- No se pueden pedir mas habitaciones de las disponibles.

Fase 2: Envio de la reserva

Si la validacion pasa, el boton cambia a 'Reservando...' y se deshabilita. Se envia una peticion POST a /api/reservas con el cuerpo en JSON: {hotel_id, nombre_cliente, fecha_entrada, fecha_salida, numero_habitaciones}.

Fase 3: Procesamiento en el backend

Flask recibe los datos, valida formatos y reglas de negocio (fechas, limites), y llama al Stored Procedure sp_realizar_reserva en MySQL:

- Verifica que el hotel existe.
- Calcula cuantas habitaciones estan ocupadas para esas fechas.
- Si hay suficientes disponibles: inserta la reserva en la tabla reservas. El trigger tr_auditoria_reservas se ejecuta automaticamente y registra la accion en auditoria_reservas con la fecha y hora actual.
- Si no hay suficientes: devuelve mensaje de error 'No hay suficientes habitaciones disponibles. Disponibles: X'.
- Devuelve el resultado: exito con ID de reserva o error con mensaje.

Fase 4: Respuesta al usuario

Si la reserva es exitosa (codigo 201): muestra un mensaje verde 'Reserva confirmada! ID: #1', actualiza la URL del navegador con la nueva informacion, y recarga el detalle del hotel para reflejar la nueva disponibilidad (las habitaciones disponibles disminuyen).

Si la reserva falla (codigo 400): muestra un mensaje rojo con el error especifico del Stored Procedure (ej: 'No hay suficientes habitaciones disponibles. Disponibles: 2').

Si hay error de conexion: muestra 'Error al conectar con el servidor'.

Finalmente, el boton se reactiva y vuelve a mostrar 'Confirmar reserva'.

phpMyAdmin (gestion de la BD)

Aparte del flujo principal, puedes acceder a http://localhost:8081 para gestionar la base de datos MySQL visualmente. Desde ahi puedes ver, insertar, modificar o eliminar registros de hoteles, reservas y auditoria sin necesidad de usar la API.